

Advanced Web Technologies



React

Basic Flutter Widgets



Flutter

Prepared by: Eng. Nidal Aljuneidi

Prerequisites

- Everything in Flutter is a widget
- Widgets are the basic building blocks of a Flutter app's UI
- They describe what their view should look like given their current configuration and state
- Widgets form a hierarchy (widget tree)
- Two types: StatelessWidget and StatefulWidget
- Flutter comes with a rich set of pre-built widgets following Material Design and Cupertino (iOS) styles



utter

Container

A convenience widget that combines common painting, positioning, and sizing

Can have padding, margins, borders, background color

Can apply transformations to its child

Single child widget

Used for:

Adding padding around widgets

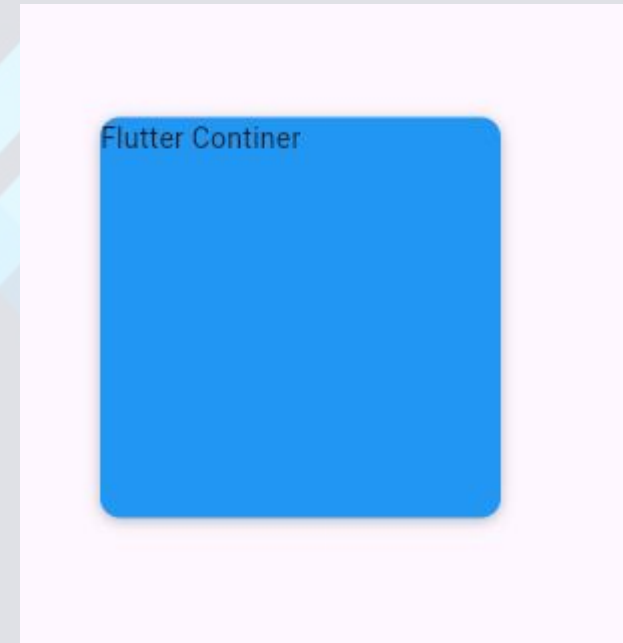
Adding backgrounds/borders

Constraining widget size



Container

```
Container(  
  width: 200,  
  height: 200,  
  margin: EdgeInsets.symmetric(vertical: 10),  
  
  decoration: BoxDecoration(  
    color: Colors.blue,  
    borderRadius: BorderRadius.circular(10),  
    boxShadow: [  
      BoxShadow(  
        color: Colors.black26,  
        blurRadius: 5,  
        offset: Offset(0, 2),  
      ),  
    ],  
  ),  
  child: Text("Flutter Continer"),  
)
```



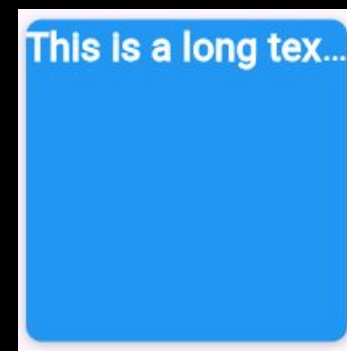
Text

- Displays a string of text with single style
- Can customize:
 - Font family, size, weight
 - Color, letter spacing
 - Text alignment
 - Text overflow behavior
- Can use rich text with TextSpan
- Supports internationalization



Flutter

```
Text(  
  'This is a long text example that might not fit in one line.',  
  style: TextStyle(  
    fontSize: 24,  
    fontWeight: FontWeight.bold,  
    color: Colors.white,  
    fontFamily: 'Roboto',  
  ),  
  textAlign: TextAlign.center,  
  overflow: TextOverflow.ellipsis,  
)
```



Rows and Columns

- Linear layout widgets
- Row: horizontal arrangement
- Column: vertical arrangement
- Control alignment with:
 - `mainAxisAlignment` (horizontal for Row, vertical for Column)
 - `crossAxisAlignment` (vertical for Row, horizontal for Column)
- `mainAxisSize` controls how much space the widget take

Rows and Columns

```
Container(  
  decoration: BoxDecoration(color: Colors.amber.shade100),  
  
  child: Row(  
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,  
    crossAxisAlignment: CrossAxisAlignment.center,  
    children: [  
      Icon(Icons.star, size: 50),  
      Text('Row Example'),  
      ElevatedButton(onPressed: () {}, child: Text('Click Me')),  
    ],  
  ),  
)
```



Row Example

Click Me



Row Example

Click Me

DEBUG

Stack

- Allows widgets to overlap
- Positions children relative to the edges of the Stack
- Can use Positioned widget to precisely place children
- Later children are drawn on top of earlier children
- Used for:
 - Overlaying text on images
 - Creating complex UIs with overlapping elements
 - Custom designs that aren't linear

Stack

```
Stack(  
  alignment: Alignment.center,  
  children: [  
    Container(width: 300, height: 200, color: Colors.blue),  
    Container(width: 150, height: 100, color: Colors.red.shade50),  
    Text(  
      'Stacked!',  
      style: TextStyle(  
        color: Colors.black,  
        fontSize: 24,  
        fontWeight: FontWeight.bold,  
      ),  
    ),  
  ],  
)
```



ListView

- Scrollable list of widgets
- Can display items vertically or horizontally
- Best for:
 - Small, finite lists
 - Lists with differently sized items
 - Static lists where all children are known
- Can separate items with Divider
- Use ListTile for standard list items



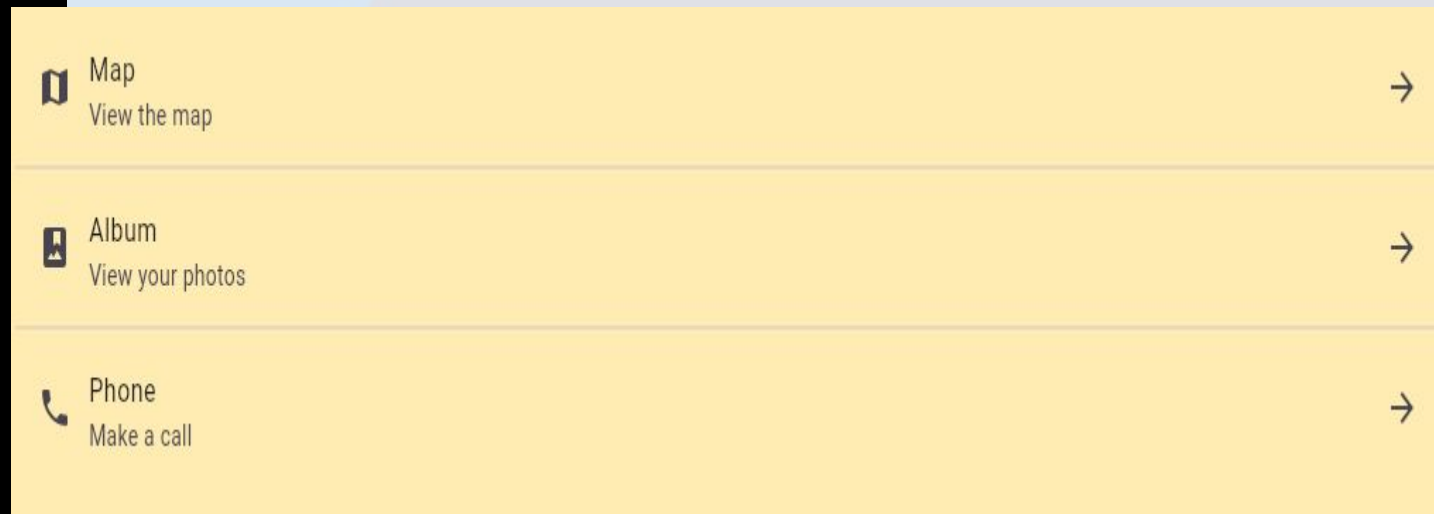
Flutter

```

child: ListView(
  padding: EdgeInsets.all(8),
  children: [
    ListTile(
      leading: Icon(Icons.map),
      title: Text('Map'),
      subtitle: Text('View the map'),
      trailing: Icon(Icons.arrow_forward),
      onTap: () {},
    ),
    Divider(),
    ListTile(
      leading: Icon(Icons.photo_album),
      title: Text('Album'),
      subtitle: Text('View your photos'),
      trailing: Icon(Icons.arrow_forward),
      onTap: () {},
    ),
    Divider(),
    ListTile(
      leading: Icon(Icons.phone),
      title: Text('Phone'),
      subtitle: Text('Make a call'),
      trailing: Icon(Icons.arrow_forward),
      onTap: () {},
    ),
  ],
),

```

ListView



ListView.builder

- Efficiently builds a scrollable list
- Builds items on demand (lazy loading)
- Great for:
 - Long or infinite lists
 - Dynamic data sources
 - Similar items with different data
- Better performance than standard ListView for long lists
- Only builds the visible items (saves memory)



Flutter

ListView.builder

```
child: Container(  
  // width: 200,  
  // height: 100,  
  decoration: BoxDecoration(color: Colors.amber.shade100),  
  child: Center(  
    child: ListView.builder(  
      itemCount: 100,  
      itemBuilder: (context, index) {  
        return ListTile(  
          leading: Icon(Icons.map),  
          title: Text('Item $index'),  
          trailing: Icon(Icons.arrow_forward),  
          onTap: () {},  
        );  
      },  
    ),  
  ),
```

Item 0

Item 1

Item 2

Item 3

Item 4

Item 5

Item 6

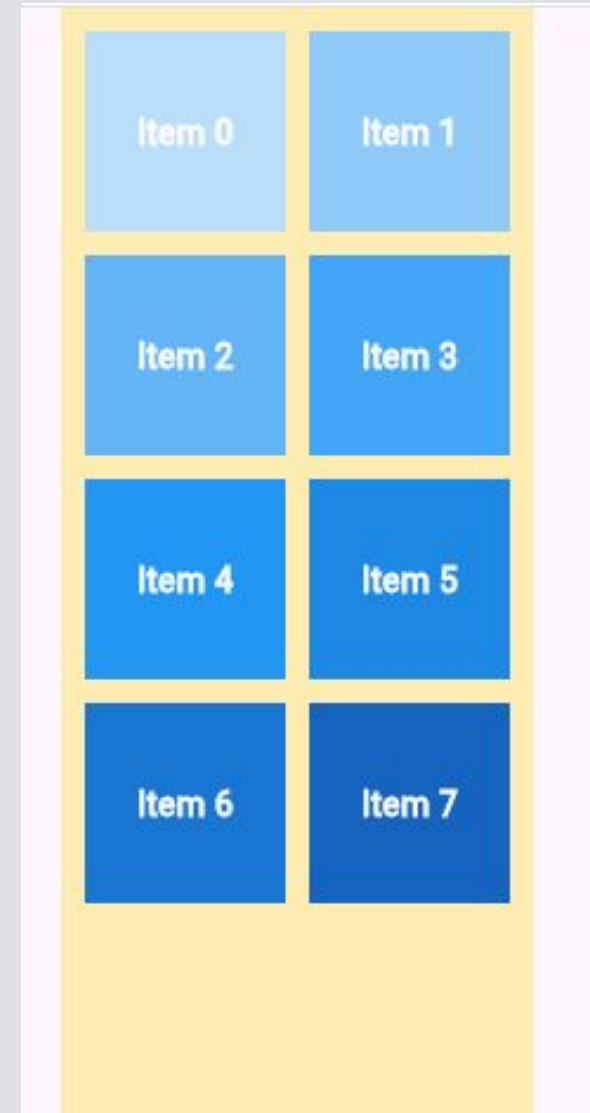


GridView

- Displays items in a 2D grid
- Scrollable grid of widgets
- Several constructors:
 - `GridView.count`: fixed number of tiles in cross axis
 - `GridView.extent`: max pixel size of tiles
 - `GridView.builder`: lazy building for large grids
- Great for photo galleries, dashboards, etc

```
GridView.count(  
    crossAxisCount: 2,  
    padding: EdgeInsets.all(10),  
    crossAxisSpacing: 10,  
    mainAxisSpacing: 10,  
    children: List.generate(8, (index) {  
        return Container(  
            color: Colors.blue[(index + 1) * 100],  
            child: Center(  
                child: Text(  
                    'Item $index',  
                    style: TextStyle(  
                        color: Colors.white,  
                        fontWeight: FontWeight.bold,  
                    ),  
                ),  
            ),  
        );  
    }  
),
```

GridView



GridView.count and GridView.builder

```
child: Center(  
  child: GridView.count(  
    crossAxisCount: 2, // Two columns  
    children: [  
      Container(color: Colors.blue, child: Text('Item 1')),  
      Container(color: Colors.red, child: Text('Item 2')),  
      Container(color: Colors.green, child: Text('Item 3')),  
      Container(color: Colors.orange, child: Text('Item 4')),  
    ],  
  ), // GridView.count  
), // Center
```



Best for **static**
and **fixed-size**
grids, Loads all
items **at once**

```
child: GridView.builder(  
  gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(  
    crossAxisCount: 2, // Two columns  
  ), // SliverGridDelegateWithFixedCrossAxisCount  
  itemCount: 100, // Large number of items  
  itemBuilder: (context, index) {  
    return Container(  
      color: Colors.blue,  
      child: Text('Item ${index + 1}'),  
    ); // Container  
  },  
), // GridView.builder
```



Up to 100

Best for
dynamic or
large grid, **Lazy**
loading

Image

- Displays images from various sources:
- **Image.network:** from URL
- **Image.asset:** from app assets
- **Image.file:** from device storage
- **Image.memory:** from memory (bytes)
- Control fit, alignment, and filtering
- Can use BoxFit to control how image fills its container
- Placeholder and error handling available



Flutter

Image

```
Image.asset(  
  'assets/images/flutter_logo.png',  
  width: 100,  
  height: 100,  
),  
Image.network(  
  'https://picsum.photos/250?image=9',  
  width: 200,  
  height: 200,  
  fit: BoxFit.cover,  
),
```



Button Widgets

- Various button types for user interaction:
- **ElevatedButton**: raised button with shadow
- **TextButton**: flat button without background
- **OutlinedButton**: button with outline
- **IconButton**: button with just an icon
- All buttons have:
 - onPressed callback for tap handling
 - customizable appearance
 - optional long press handling



Flutter

```

Row(
  crossAxisAlignment: CrossAxisAlignment.center,
  mainAxisAlignment: MainAxisAlignment.spaceAround,
  children: [
    // Elevated Button (raised)
    ElevatedButton(
      onPressed: () {},
      child: Text('Elevated Button'),
    ),

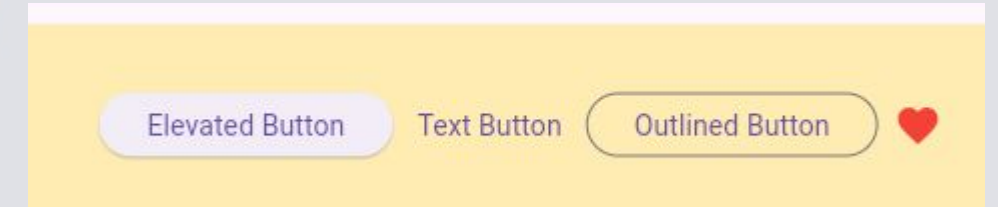
    // Text Button (flat)
    TextButton(onPressed: () {}, child: Text('Text Button')),

    // Outlined Button
    OutlinedButton(
      onPressed: () {},
      child: Text('Outlined Button'),
    ),

    // IconButton
    IconButton(
      icon: Icon(Icons.favorite),
      color: Colors.red,
      onPressed: () {},
    ),
  ],
),

```

Button Widgets



Card

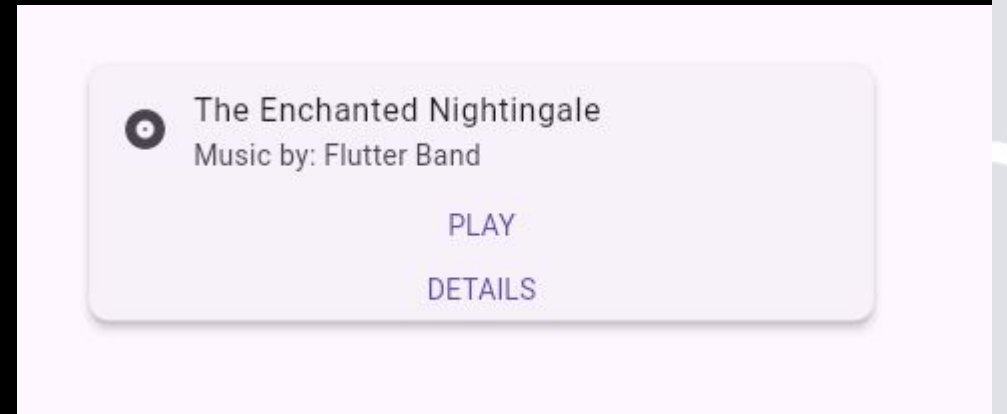
- Material Design card
- Rounded corners and elevation
- Used to group related content
- Can contain various widgets
- Best practices:
 - Keep content focused
 - Use for interactive elements
 - Provide clear actions
- Commonly used in feeds, dashboards, and lists



Flutter

Card

```
Card(  
  elevation: 4,  
  shape: RoundedRectangleBorder(  
    borderRadius: BorderRadius.circular(10),  
  ),  
  child: Column(  
    mainAxisAlignment: MainAxisAlignment.min,  
    children: [  
      ListTile(  
        leading: Icon(Icons.album),  
        title: Text(''),  
        subtitle: Text('Music by: Flutter Band'),  
      ),  
      TextButton(onPressed: () {}, child: Text('PLAY')),  
      TextButton(onPressed: () {}, child: Text('DETAILS')),  
    ],  
  ),  
),
```



App Bar

- Material Design app bar
- Typically used as the top bar in Scaffold
- Features:
 - Title and optional subtitle
 - Leading widget (typically back or menu button)
 - Action buttons
 - Bottom widgets (like tabs)
 - Flexible space for expandable headers
- Can be customized with different colors, elevation



Flutter

App Bar

```
appBar: AppBar(  
  title: Text('Flutter App'),  
  backgroundColor: Colors.blue,  
  
  leading: IconButton(icon: Icon(Icons.menu), onPressed: () {}),  
  actions: [  
    IconButton(icon: Icon(Icons.search), onPressed: () {}),  
    IconButton(icon: Icon(Icons.more_vert), onPressed: () {}),  
  ],  
),
```

≡ Flutter App



Scaffold

- Material Design basic page structure
- Provides API for showing:
 - AppBar at the top
 - Floating action button
 - Drawers (side menu)
 - Bottom navigation bar
 - Snack bars (temporary messages)
 - Bottom sheets
- The body property contains the main content
- Most Flutter apps use Scaffold as the root layout



Flutter

```
home: Scaffold(  
  appBar: AppBar(title: Text('My Flutter App')),  
  body: Center(child: Text('Hello, Flutter!')),  
  floatingActionButton: FloatingActionButton(  
    onPressed: () {},  
    child: Icon(Icons.add),  
  ),  
  drawer: Drawer(  
    child: ListView(  
      padding: EdgeInsets.zero,  
      children: [  
        DrawerHeader(  
          decoration: BoxDecoration(color: Colors.blue),  
          child: Text(  
            'Drawer Header',  
            style: TextStyle(color: Colors.white, fontSize: 24),  
          ),  
        ),  
        ListTile(title: Text('Item 1'), onTap: () {}),  
        ListTile(title: Text('Item 2'), onTap: () {}),  
      ],  
    ),  
  ),  
),
```

Flutter

```
bottomNavigationBar: BottomNavigationBar(  
  items: [  
    BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),  
    BottomNavigationBarItem(  
      icon: Icon(Icons.business),  
      label: 'Business',  
    ),  
    BottomNavigationBarItem(icon: Icon(Icons.school), label: 'School'),  
  ],  
  currentIndex: 0,  
  onTap: (index) {},  
),  
)
```

```
bottomNavigationBar: BottomNavigationBar(  
  items: [  
    BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),  
    BottomNavigationBarItem(  
      icon: Icon(Icons.business),  
      label: 'Business',  
    ),  
    BottomNavigationBarItem(icon: Icon(Icons.school), label: 'School'),  
  ],  
  currentIndex: 0,  
  onTap: (index) {},  
)
```

☰ My Flutter App

Hello, Flutter!



Home



Business



School

Drawer Header

Item 1

Item 2

Hello, Flutter!



Business



School



React



Flutter

```
import 'package:flutter/material.dart';

void main(List<String> args) {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedBanner: false,
      home: Scaffold(
        appBar: AppBar(title: Text('My Flutter App')),
        body: Center(child: Text('Hello, Flutter!')),
        floatingActionButton: FloatingActionButton(
          onPressed: () {},
          child: Icon(Icons.add),
        ),
        drawer: Drawer(
          child: ListView(
            padding: EdgeInsets.zero,
            children: [
              DrawerHeader(
                decoration: BoxDecoration(color: Colors.blue),
                child: Text(
                  'Drawer Header',
                  style: TextStyle(color: Colors.white, fontSize: 24),
                ),
              ),
              ListTile(title: Text('Item 1'), onTap: () {}),
              ListTile(title: Text('Item 2'), onTap: () {}),
            ],
          ),
        ),
        bottomNavigationBar: BottomNavigationBar(
          items: [
            BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),
            BottomNavigationBarItem(
              icon: Icon(Icons.business),
              label: 'Business',
            ),
            BottomNavigationBarItem(icon: Icon(Icons.school), label: 'School'),
          ],
          currentIndex: 0,
          onTap: (index) {},
        ),
      ),
    );
  }
}
```

Introduction to Routing in Flutter

Routing allows navigation between screens in a Flutter app

- Supports smooth transitions, back navigation, and dynamic content
- Used for page navigation in app

Types of Routing in Flutter

Navigator.push → Directly moves to a new screen

Navigator.pushNamed → Uses predefined names for cleaner navigation

onGenerateRoute → Handles dynamic routes

GoRouter Package → Advanced navigation

Flutter

Basic Navigation (**Navigator.push**)

This method **manually creates screens** and navigates between them.

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: HomeScreen(),  
    );  
  }  
}
```

Flutter

Home Screen

```
class HomeScreen extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: Text("Home Screen")),  
      body: Center(  
        child: ElevatedButton(  
          onPressed: () {  
            Navigator.push(  
              context,  
              MaterialPageRoute(builder: (context) => SecondScreen()), // Navigate to SecondScreen  
            );  
          },  
          child: Text("Go to Second Screen"),  
        ),  
      ),  
    );  
  }  
}
```

- * **Pushes** a new screen onto the navigation stack
- * Back navigation using `Navigator.pop(context);`

Second Screen

```
class SecondScreen extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: Text("Second Screen")),  
      body: Center(  
        child: ElevatedButton(  
          onPressed: () {  
            Navigator.pop(context); // Go back to HomeScreen  
          },  
          child: Text("Go Back"),  
        ),  
      ),  
    );  
  }  
}
```

Named Routes (Navigator.pushNamed)

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      initialRoute: '/',  
      routes: {  
        '/': (context) => HomeScreen(),  
        '/second': (context) => SecondScreen(),  
      },  
    );  
  }  
}
```

This method
predefines routes
for structured
navigation

Home Screen

```
class HomeScreen extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: Text("Home Screen")),  
      body: Center(  
        child: ElevatedButton(  
          onPressed: () {  
            Navigator.pushNamed(context, '/second'); // Navigate to SecondScreen  
          },  
          child: Text("Go to Second Screen"),  
        ),  
      ),  
    );  
  }  
}
```

Second Screen

```
class SecondScreen extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: Text("Second Screen")),  
      body: Center(  
        child: ElevatedButton(  
          onPressed: () {  
            Navigator.pop(context); // Return to HomeScreen  
          },  
          child: Text("Go Back"),  
        ),  
      ),  
    );  
  }  
}
```

Question

Build a multi-screen Flutter app that explores different Palestinian cities , using:

- Named routes
- A navigation drawer
- A bottom navigation bar
- Basic layout and media widgets

Each screen will showcase one Palestinian city using various Flutter widgets like

Container, Text, Row, Column, Stack, ListView, ListView.builder, GridView, Buttons, Image, Card, AppBar, and Scaffold.

MaterialApp

main.dart

```
MaterialApp(  
  title: 'Palestine Cities Explorer',  
  theme: ThemeData(primarySwatch: Colors.green),  
  initialRoute: '/',  
  routes: {  
    '/': (context) => const HomeScreen(),  
    '/jerusalem': (context) => const JerusalemScreen(),  
    '/bethlehem': (context) => const BethlehemScreen(),  
    '/hebron': (context) => const HebronScreen(),  
    '/nablus': (context) => const NablusScreen(),  
  },  
);
```

home_screedart

```
import 'package:flutter/material.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({Key? key}) : super(key: key);

  @override
  State<HomeScreen> createState() => HomeScreenState();
}

class HomeScreenState extends State<HomeScreen> {
  int _selectedIndex = 0;

  final List<String> _routes = [
    '/',
    '/jerusalem',
    '/bethlehem',
    '/hebron',
    '/nablus',
  ];

  final List<String> _titles = [
    'Welcome',
    'Jerusalem',
    'Bethlehem',
    'Hebron',
    'Nablus',
  ];

  void _onItemTapped(int index) {
    setState(() {
      _selectedIndex = index;
    });
    Navigator.pushNamed(context, _routes[index]);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(_titles[_selectedIndex])),
      drawer: Drawer(
        child: ListView(
          padding: EdgeInsets.zero,
          children: [
            const DrawerHeader(
              decoration: BoxDecoration(color: Colors.green),
              child: Text(
                'Explore Palestinian Cities',
                style: TextStyle(color: Colors.white, fontSize: 20),
              ),
            ),
            ListTile(
              leading: const Icon(Icons.location_city),
              title: const Text('Jerusalem'),
              onTap: () => Navigator.pushNamed(context, '/jerusalem'),
            ),
            ListTile(
              leading: const Icon(Icons.nature_people),
              title: const Text('Bethlehem'),
              onTap: () => Navigator.pushNamed(context, '/bethlehem'),
            ),
            ListTile(
              leading: const Icon(Icons.place),
              title: const Text('Hebron'),
              onTap: () => Navigator.pushNamed(context, '/hebron'),
            ),
            ListTile(
              leading: const Icon(Icons.map),
              title: const Text('Nablus'),
              onTap: () => Navigator.pushNamed(context, '/nablus'),
            ),
          ],
        ),
      ),
      body: Center(child: Text('Welcome to Palestine Cities Explorer')),
      bottomNavigationBar: BottomNavigationBar(
        currentIndex: _selectedIndex,
```



Flutter



localhost:61401



Explore Palestinian Cities



Jerusalem



Bethlehem



Hebron



Nablus

Welcome to Palestine Cities Explorer

DEBUG